



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Lab experiment-7

Breadth First Search (BFS)

Aim

To write a Python program to implement the Breadth First Search (BFS) algorithm.

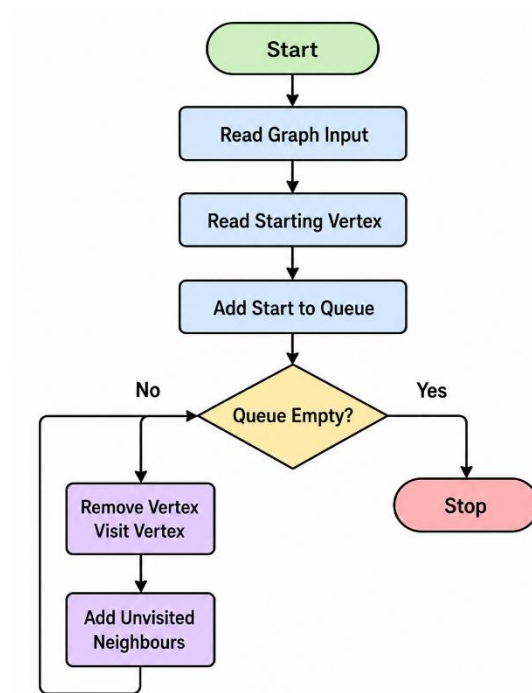
Objective

To traverse a graph using the BFS algorithm.

Algorithm

1. Start.
2. Read the number of vertices and edges.
3. Create the graph using an adjacency list.
4. Read the starting vertex.
5. Mark the starting vertex as visited and add it to the queue.
6. Remove a vertex from the queue, visit it, and add all unvisited adjacent vertices.
7. Repeat until the queue becomes empty.
8. Stop.

Flowchart



Code

```
from collections import deque
```

```
graph = {}
```

```
n = int(input("Enter number of vertices: "))
```

```
for _ in range(n):
```

```
    v = input("Enter vertex: ")
```

```
    graph[v] = input(f"Enter neighbours of  
{v} (space-separated): ").split()
```

```
start = input("Enter starting vertex: ")
```

```
visited = set([start])
```

```
queue = deque([start])
```

```
print("BFS Traversal:", end=" ")
```

```
while queue:
```

```
    node = queue.popleft()
```

```
    print(node, end=" ")
```

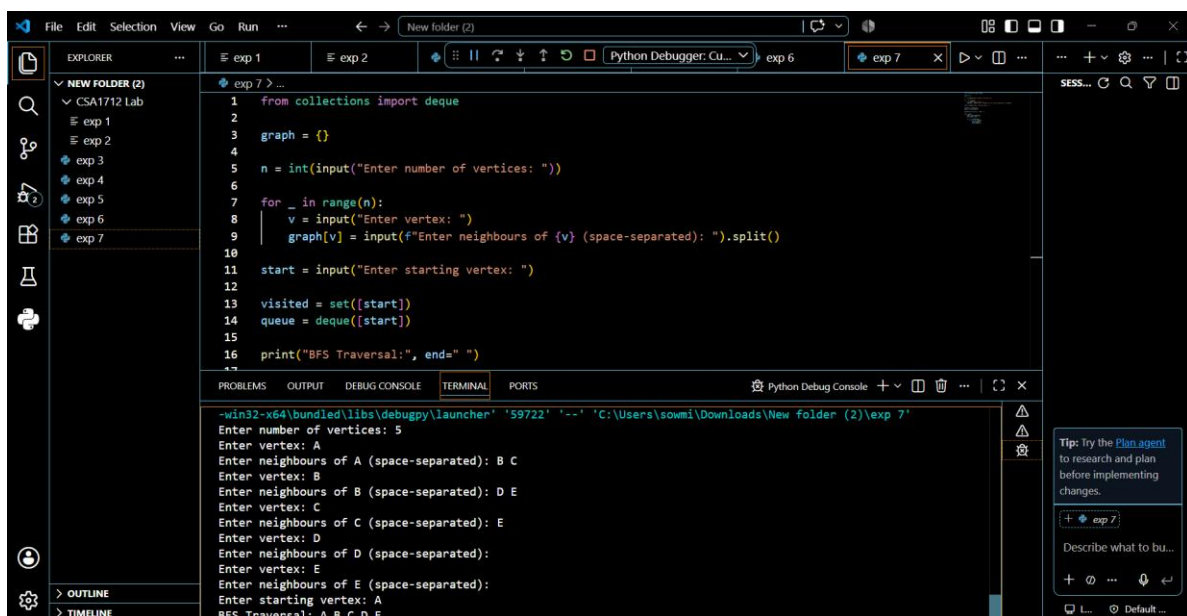
```
    for i in graph[node]:
```

```
        if i not in visited:
```

```
            visited.add(i)
```

```
            queue.append(i)
```

Output



The screenshot shows a Python IDE with a dark theme. The Explorer panel on the left shows a project named 'NEW FOLDER (2)' containing a subfolder 'CSA1712 Lab' with files 'exp 1' through 'exp 7'. The main editor displays a Python script for BFS Traversal. The script prompts the user for the number of vertices (5), then for each vertex (A, B, C, D, E) and its neighbors. The output shows the BFS traversal order: A B C D E.

```
1 from collections import deque
2
3 graph = {}
4
5 n = int(input("Enter number of vertices: "))
6
7 for _ in range(n):
8     v = input("Enter vertex: ")
9     graph[v] = input(f"Enter neighbours of {v} (space-separated): ").split()
10
11 start = input("Enter starting vertex: ")
12
13 visited = set([start])
14 queue = deque([start])
15
16 print("BFS Traversal:", end=" ")
17
```

Terminal Output:

```
-win32-x64\bundled\libs\debugpy\launcher '59722' '-' 'C:\Users\sowmi\Downloads\New folder (2)\exp 7'
Enter number of vertices: 5
Enter vertex: A
Enter neighbours of A (space-separated): B C
Enter vertex: B
Enter neighbours of B (space-separated): D E
Enter vertex: C
Enter neighbours of C (space-separated): E
Enter vertex: D
Enter neighbours of D (space-separated):
Enter vertex: E
Enter neighbours of E (space-separated):
Enter starting vertex: A
BFS Traversal: A B C D E
```

Result

The graph was traversed successfully using the BFS algorithm.

Conclusion

The BFS algorithm successfully visits all reachable vertices level by level using a queue.